

# RPi5 카메라 실습

## 실습 교재

CSI 카메라 · Picamera2 · OpenCV · YOLO v11 · NCNN 경량 추론  
중앙대학교 전자전기공학부

### 📖 교재 구성

본 교재는 Raspberry Pi 5에서 CSI 카메라를 연결하고 Python Picamera2 라이브러리로 제어하는 방법부터, OpenCV를 이용한 실시간 영상 처리, 그리고 YOLO v11 딥러닝 모델을 이용한 실시간 객체 탐지까지 체계적으로 설명합니다. 모든 예제 코드는 Raspberry Pi OS Bookworm(Debian 12) 기준으로 작성되었습니다.

| 장 | 주제                         | 주요 내용                                                                                |
|---|----------------------------|--------------------------------------------------------------------------------------|
| 1 | CSI 카메라 연결 및 기본 제어         | rpicam 명령어, rpicam-still, rpicam-vid, Picamera2 API, 동영상·사진 촬영                       |
| 2 | Python 가상환경 및 OpenCV 설치    | python -m venv, --system-site-packages, activate, pip3 install opencv-contrib-python |
| 3 | OpenCV 기본 이미지 처리           | imread/imshow/resize/flip, BGR 채널 순서, cvtColor, hconcat                              |
| 4 | OpenCV + Picamera2 실시간 카메라 | capture_array, while 루프 미리보기, waitKey, 's' 키 사진 저장                                   |
| 5 | YOLO v11 설치 및 실시간 객체 탐지    | ultralytics, PyTorch, NCNN 변환, FPS 측정, results.plot(), box.cls/conf                  |

### 참고 문헌

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤써북, 2024.

이현·김경연·김명생, 라즈베리파이5로 구현하는 사물인터넷(IoT)과 인공지능(AI) 자율주행 자동차, 광문각, 2025.

# 제 1장 CSI 카메라 연결 및 기본 제어

## 1.1 RPi5 CSI 카메라 모듈

Raspberry Pi 5는 보드 상단에 두 개의 CSI(Camera Serial Interface) 포트를 내장하고 있습니다. CSI는 카메라 센서와 SoC를 직접 연결하는 고속 직렬 인터페이스로, USB 카메라 대비 낮은 레이턴시와 높은 해상도를 지원합니다. 주요 카메라 모듈은 Camera Module v2(8MP, 고정 초점)와 v3(12MP, 자동 초점)이며, 그 외 HQ 카메라(12.3MP, C/CS 마운트 교환 렌즈 지원)도 사용할 수 있습니다.

Raspberry Pi 4까지는 15핀 FFC(Flexible Flat Cable) 케이블을 사용했으나, RPi5부터는 더 작은 22핀 FFC 케이블로 변경되었습니다. 카메라 모듈 패키지에는 15핀·22핀 케이블이 모두 동봉되므로, RPi5에 연결할 때는 반드시 22핀 케이블을 사용합니다.

⚠ RPi5의 CSI 커넥터 잠금 클립은 RPi4와 방향이 반대입니다. 클립을 위로 당겨 열고 케이블의 금속 접점이 PCB 쪽을 향하도록 삽입한 뒤, 클립을 아래로 눌러 고정합니다.

## 1.2 카메라 동작 확인

카메라를 연결하고 전원을 켜면 Raspberry Pi OS Bookworm은 libcamera 스택을 통해 자동 인식합니다. 동작 확인은 `rpicam` 계열 명령어를 사용합니다.

🔗 Raspberry Pi OS Bookworm(Debian 12)부터 기존 `libcamera-still`, `libcamera-vid` 등의 명령어가 `rpicam-still`, `rpicam-vid`로 이름이 변경되었습니다. 이전 교재나 인터넷 자료에 `libcamera-***` 형태로 나오면 `rpicam-***`으로 바꿔 실행하십시오.

### 1.2.1 연결된 카메라 목록 확인

아래 명령을 실행하면 현재 연결된 카메라의 인덱스, 모델명, 지원 해상도·포맷 목록이 출력됩니다.

```
$ rpicam-hello --list-cameras
```

정상적으로 인식된 경우 아래와 유사한 출력이 나타납니다.

```
Available cameras
-----
0 : imx708 [4608x2592 12-bit RGGB] (/base/.../imx708@1a)
    Modes: 'SRGGB10_CSI2P' : 1536x864 [120.13 fps] ...
```

💡 팁: 카메라가 'No cameras available'로 표시되면 ① 22핀 케이블 방향 확인, ② `sudo raspi-config` → `Interface Options` → `Legacy Camera` 설정 확인, ③ 재부팅 순서로 점검합니다.

### 1.2.2 5초 미리보기

`rpicam-hello`를 인자 없이 실행하면 기본 5초 동안 카메라 미리보기 창이 표시됩니다. GUI 환경(데스크탑

모드)에서 실행해야 하며, SSH 텍스트 세션에서는 표시되지 않습니다.

```
$ rpicam-hello
```

## 1.3 사진 및 동영상 촬영

### 1.3.1 사진 촬영 (rpicam-still)

rpicam-still은 정지 이미지를 캡처합니다. -o 옵션으로 저장 파일명을 지정하며, 파일 확장자(jpg/png/bmp)에 따라 포맷이 자동 결정됩니다.

```
$ rpicam-still -o image.jpg

# 1920x1080 고해상도로 촬영
$ rpicam-still -o image.jpg --width 1920 --height 1080

# 즉시 촬영 (대기 시간 500ms)
$ rpicam-still -o image.jpg -t 500

# 좌우 반전 + 품질 95
$ rpicam-still -o image.jpg --hflip -q 95
```

| 옵션                 | 설명                         |
|--------------------|----------------------------|
| -o <파일명>           | 저장 파일명 지정 (확장자로 포맷 자동 결정)  |
| --width / --height | 캡처 해상도 지정 (픽셀 단위)          |
| -t <ms>            | 촬영까지 대기 시간(ms). 기본값 5000ms |
| -q <0~100>         | JPEG 압축 품질. 기본값 93         |
| --hflip / --vflip  | 수평 / 수직 반전                 |
| -n                 | 미리보기 창 표시 안 함 (헤드리스 환경)    |

### 1.3.2 동영상 촬영 (rpicam-vid)

rpicam-vid는 동영상을 H.264 원시 스트림으로 녹화합니다. 일반 미디어 플레이어에서 재생하려면 ffmpeg으로 MP4 컨테이너로 감싸야 합니다.

```
# 기본 녹화 (5초, H.264)
$ rpicam-vid -o video.h264

# 10초간 1080p 30fps 녹화
$ rpicam-vid -o video.h264 -t 10000 --width 1920 --height 1080 --framerate 30

# H.264 → MP4 변환 (ffmpeg 필요)
$ ffmpeg -r 30 -i video.h264 -vcodec copy video.mp4
```

## 1.4 Python Picamera2 라이브러리

Picamera2는 libcamera 스택 기반의 공식 Python 카메라 라이브러리로, 이전의 picamera(v1)를 대체합니다. Raspberry Pi OS Bookworm에는 기본 설치되어 있으며, 없으면 아래 명령으로 설치합니다.

```
$ sudo apt install python3-picamera2
```

### 1.4.1 동영상 녹화 (picam.py)

미리보기 창을 표시하면서 5초간 동영상을 녹화하는 기본 예제입니다. Preview.QTGL은 GPU 가속 Qt/OpenGL 기반 미리보기를 사용합니다.

```
# picam.py
from picamera2 import Picamera2, Preview

picam2 = Picamera2()
camera_config = picam2.create_preview_configuration()
picam2.configure(camera_config)
picam2.start_preview(Preview.QTGL) # 미리보기 창 열기
picam2.start_and_record_video("cos.h264", duration=5) # 5초 녹화 후 자동 종료
$ python3 picam.py
```

start\_and\_record\_video()는 미리보기와 녹화를 동시에 시작하고, duration 초 후 자동으로 카메라를 정지합니다. 완료 후 현재 디렉토리에 cos.h264 파일이 생성됩니다.

### 1.4.2 사진 촬영 (picapture.py)

카메라를 2초간 안정화한 뒤 현재 프레임을 JPEG 파일로 저장합니다. capture\_file()은 가장 간단하게 프레임을 파일로 저장하는 메서드입니다.

```
# picapture.py
from picamera2 import Picamera2, Preview
import time

picam2 = Picamera2()
camera_config = picam2.create_preview_configuration()
picam2.configure(camera_config)
picam2.start_preview(Preview.QTGL)
picam2.start() # 카메라 스트리밍 시작 (녹화 없음)
time.sleep(2) # AEC/AWB 자동 수렴 대기
picam2.capture_file("cos.jpg") # 현재 프레임을 JPEG로 저장
$ python3 picapture.py
```

💡 *팁: time.sleep(2)는 자동 노출(AEC)과 자동 화이트밸런스(AWB)가 안정적인 값으로 수렴하는 대기 시간입니다. 생략하면 어둡거나 색 편차가 있는 사진이 촬영됩니다.*

### 1.4.3 미리보기 위치·크기·방향 지정 (picam2.py)

start\_preview()에 x, y, width, height와 Transform 객체를 전달하여 미리보기 창의 표시 방식을 세밀하게 제어할 수 있습니다.

```
# picam2.py
from picamera2 import Picamera2, Preview
from libcamera import Transform
import time

picam2 = Picamera2()
picam2.start_preview(
```

```

Preview.QTGL,
x=100, y=200,          # 화면에서 창을 띄울 위치 (픽셀)
width=800, height=600, # 미리보기 창 크기
transform=Transform(vflip=1) # 수직 반전
)
picam2.start()
time.sleep(10)    # 10초간 유지 (이 없으면 창이 즉시 닫힘)
picam2.stop()
$ python3 picam2.py

```

⚠ `start()` 호출 후 `time.sleep()` 없이 스크립트가 바로 종료되면 미리보기 창이 즉시 닫힙니다. 미리보기를 유지하려면 반드시 대기 구문을 추가하십시오.

🔗 `Transform` 옵션: `vflip=1`(상하 반전), `hflip=1`(좌우 반전). 카메라 모듈이 거꾸로 장착된 경우 `vflip=1`을 적용합니다. `rotation=90` 옵션은 일부 센서에서만 지원됩니다.

#### 1.4.4 Picamera2 주요 API 정리

| API                                                                                                                                         | 설명                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Picamera2()</b><br><b>create_preview_configuration()</b>                                                                                 | Picamera2 인스턴스 생성 및 카메라 하드웨어 초기화<br>미리보기용 설정 생성. <code>main={'format':..., 'size':...}</code> 로 세부 지정 가능                                                 |
| <b>configure(config)</b>                                                                                                                    | 생성한 설정을 카메라에 적용. <code>start()</code> 전에 반드시 호출                                                                                                          |
| <b>start_preview(Preview.QTGL, ...)</b><br><br><b>start()</b><br><b>start_and_record_video(파일명, duration=N)</b><br><b>capture_file(파일명)</b> | Qt/OpenGL GPU 가속 미리보기 창 열기. 위치·크기·Transform 지정 가능<br>카메라 스트리밍 시작 (화면 표시·녹화 없음)<br>미리보기 + 녹화를 동시에 시작. N초 후 자동 종료<br>현재 프레임을 JPEG/PNG 파일로 저장. 가장 간단한 저장 방식 |
| <b>capture_array()</b><br><br><b>stop()</b>                                                                                                 | 현재 프레임을 NumPy 배열로 반환. OpenCV 연동 시 핵심 메서드<br>카메라 스트리밍 정지 및 리소스 해제                                                                                         |

## 제 2장 Python 가상환경 및 OpenCV 설치

### 2.1 Python 가상환경(venv) 개요

Python 가상환경(Virtual Environment)은 프로젝트마다 독립적인 패키지 공간을 만드는 도구입니다. 가상환경을 사용하면 프로젝트A에는 OpenCV 4.8을, 프로젝트B에는 OpenCV 4.13을 충돌 없이 따로 설치할 수 있으며, 시스템 Python 환경을 오염시키지 않습니다.

Raspberry Pi OS Bookworm(Debian 12)부터는 pip를 이용한 시스템 전역 패키지 직접 설치가 제한됩니다. OpenCV, PyTorch 같은 서드파티 패키지는 가상환경 안에서 설치하는 것이 권장됩니다.

### 2.2 가상환경 생성 및 활성화

#### 2.2.1 가상환경 생성

--system-site-packages 옵션을 추가하면 시스템에 이미 설치된 python3-picamera2, python3-rpi-gpio 등을 가상환경 안에서도 그대로 사용할 수 있습니다. 이 옵션 없이 생성하면 가상환경은 아무 패키지도 없는 빈 상태로 시작합니다.

```
# 홈 디렉토리에 camera_lab 폴더 생성 후 이동
$ mkdir ~/camera_lab && cd ~/camera_lab

# --system-site-packages 옵션으로 가상환경 생성
$ python -m venv --system-site-packages venv

# 생성 확인
$ ls
... venv ...
```

#### 2.2.2 가상환경 활성화

가상환경을 사용하려면 반드시 활성화해야 합니다. 활성화 후에는 터미널 프롬프트 앞에 (venv)가 붙어 현재 가상환경이 활성화되어 있음을 나타냅니다. 이후 모든 pip 설치하는 이 가상환경 안에만 적용됩니다.

```
$ source venv/bin/activate

(venv) pi@raspberrypi:~/camera_lab$
```

가상환경을 종료하려면 deactivate를 실행합니다.

```
(venv) $ deactivate
```

💡 *참: SSH 재접속 시마다 source venv/bin/activate를 실행해야 합니다. ~/.bashrc 마지막 줄에 추가하거나, 실행용 셸 스크립트(run.sh)로 만들어두면 편리합니다.*

## 2.3 OpenCV 설치 및 확인

OpenCV(Open Source Computer Vision Library)는 실시간 이미지·동영상 처리를 위한 오픈소스 라이브러리입니다. `opencv-contrib-python` 패키지는 기본 `opencv-python`에 추가 기여 모듈(SIFT, 추적 알고리즘 등)을 더한 풀 버전입니다.

```
# 가상환경이 활성화된 상태에서 실행
(venv) $ pip3 install opencv-contrib-python
```

설치 완료 후 Python에서 `cv2` 모듈을 임포트하여 버전을 확인합니다.

```
(venv) $ python3
>>> import cv2
>>> cv2.__version__
'4.13.0'
>>> exit()
```

## 2.4 실습 디렉토리 구조

이후 모든 실습 파일을 아래 구조로 준비합니다.

```
~/camera_lab/
├── venv/                # Python 가상환경
├── img_1.png            # OpenCV 테스트용 이미지
├── picam.py             # Picamera2 동영상 녹화
├── picapture.py         # Picamera2 사진 촬영
├── cv4_1.py             # OpenCV 이미지 파일 표시
├── cv4_3.py             # OpenCV 카메라 미리보기
├── cv4_4.py             # OpenCV 키 입력 사진 저장
├── convert2ncnn.py      # YOLO → NCNN 변환
├── yolov11ncnn-test.py  # NCNN 실시간 객체 탐지
└── yolo11n_ncnn_model/  # NCNN 변환 완료 모델
```

## 제 3장 OpenCV 기본 이미지 처리

### 3.1 OpenCV 이미지 데이터 구조

OpenCV는 이미지를 NumPy 배열(ndarray)로 처리합니다. 컬러 이미지는 (높이 H, 너비 W, 채널 수 3) 형태의 3차원 배열이며, 픽셀값은 0~255 범위의 uint8 정수입니다. 중요한 특징은 OpenCV가 채널을 RGB 순서가 아닌 BGR(파란색·초록색·빨간색) 순서로 저장한다는 점입니다. Matplotlib이나 다른 라이브러리와 연동 시 `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`로 변환해야 합니다.

#### 3.1.1 OpenCV 핵심 함수 정리

| 함수                                                                                                                 | 설명                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cv2.imread(경로)</code>                                                                                        | 이미지 파일을 읽어 NumPy 배열 반환. 파일 없으면 None 반환                                                                                                                                   |
| <code>cv2.imshow(창이름, 이미지)</code>                                                                                  | 이름 있는 창에 이미지 표시. <code>waitKey()</code> 없이는 즉시 닫힘                                                                                                                        |
| <code>cv2.imwrite(경로, 이미지)</code>                                                                                  | NumPy 배열을 이미지 파일로 저장. 확장자로 포맷 결정                                                                                                                                         |
| <code>cv2.resize(img, (w, h))</code><br><code>cv2.flip(img, flipCode)</code><br><code>cv2.cvtColor(img, 코드)</code> | 이미지 크기 변경. 두 번째 인자는 (너비, 높이) 순서에 주의<br>반전. 0=상하, 1=좌우, -1=상하+좌우 동시<br>색 공간 변환. 예: <code>COLOR_BGR2GRAY</code> , <code>COLOR_BGR2RGB</code> , <code>COLOR_GRAY2BGR</code> |
| <code>cv2.waitKey(ms)</code><br><code>cv2.destroyAllWindows()</code>                                               | 키 입력 대기. 0이면 무한 대기. 반환값: 눌린 키의 ASCII 코드<br>열린 모든 OpenCV 창 닫기                                                                                                             |
| <code>cv2.putText(img, 텍스트, 위치, ...)</code>                                                                        | 이미지 위에 텍스트 오버레이. 탐지 결과 표시에 활용                                                                                                                                            |
| <code>cv2.hconcat([img1, img2])</code>                                                                             | 두 이미지를 가로로 연결. 비교 표시에 유용                                                                                                                                                 |

### 3.2 이미지 파일 표시 (cv4\_1.py)

이미지 파일을 읽어 크기를 조정하고 창에 표시하는 기본 예제입니다. 아무 키나 누르면 창이 닫히고 프로그램이 종료됩니다.

```
# cv4_1.py
import cv2

img_file = "./img_1.png"      # 이미지 파일 경로
img = cv2.imread(img_file)     # NumPy 배열로 읽기 (BGR 채널 순서)

if img is not None:
    img_resize = cv2.resize(img, (960, 540)) # (너비, 높이) 순서 주의
    cv2.imshow("IMG", img_resize)           # 창 이름: "IMG"
    cv2.waitKey()                           # 아무 키나 누를 때까지 대기
    cv2.destroyAllWindows()
else:
    print("Image file not found")

(venv) $ python3 cv4_1.py
```



### 3.2.1 코드 주요 포인트 해설

**cv2.imread()** 반환값: 성공 시 NumPy ndarray, 실패(파일 없음 등) 시 None. 항상 None 체크를 습관화합니다.

**cv2.resize(img, (너비, 높이))**: dsize 인자는 (너비, 높이) 순서입니다. **img.shape**는 (높이, 너비, 채널) 순서이므로 반대임을 주의합니다.

**cv2.waitKey()**: 인자 없이 호출하면 0을 의미하며 무한 대기합니다. 반환값은 눌린 키의 ASCII 코드입니다. 양수를 전달하면 해당 ms 후 자동 반환됩니다.

읽어온 이미지의 크기·타입을 확인하는 방법은 다음과 같습니다.

```
print(img.shape)    # 출력 예: (1080, 1920, 3) → (높이, 너비, 채널)
print(img.dtype)    # 출력 예: uint8
print(img.size)     # 전체 원소 수 = 높이×너비×채널
```

### 3.2.2 그레이스케일 변환 및 원본 비교 표시

원본 컬러 이미지와 그레이스케일 변환 이미지를 나란히 표시하는 확장 예제입니다.

```
# cv4_1_ext.py
import cv2

img = cv2.imread("./img_1.png")
if img is not None:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # 그레이스케일 변환

    # 같은 크기로 리사이즈
    img_r = cv2.resize(img, (640, 360))
    gray_r = cv2.resize(gray, (640, 360))

    # hconcat은 채널 수가 같아야 함 → 그레이를 3채널로 변환
    gray_bgr = cv2.cvtColor(gray_r, cv2.COLOR_GRAY2BGR)

    combined = cv2.hconcat([img_r, gray_bgr])
    cv2.imshow("Original | Grayscale", combined)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

🔗 **cv2.COLOR\_GRAY2BGR**은 1채널 그레이 이미지를 3채널(동일값 복사)로 변환합니다. **hconcat**은 두 배열의 채널 수가 일치해야 하므로 이 변환이 필요합니다.

## 제 4장 OpenCV + Picamera2 실시간 카메라 제어

### 4.1 Picamera2 + OpenCV 연동 구조

Picamera2의 `capture_array()`는 카메라 프레임을 NumPy 배열로 반환합니다. 이 배열을 그대로 OpenCV 함수에 전달하여 처리하면 Picamera2 캡처 → OpenCV 처리 → `cv2.imshow` 표시 파이프라인을 구성할 수 있습니다.

한 가지 주의 사항은 Picamera2 기본 포맷인 XRGB8888의 채널 순서(X, R, G, B)가 OpenCV가 기대하는 BGR과 다르다는 점입니다. 실습에서는 미리보기 설정 시 `format`을 'XRGB8888'로 지정하고 OpenCV에서 그대로 사용하거나, 'RGB888'로 지정하고 필요 시 `cvtColor`로 변환합니다.

```
# 포맷을 'XRGB8888'로 설정하는 방식 (가장 일반적)
preview_config = picam2.create_preview_configuration(
    main={"format": "XRGB8888", "size": (640, 480)}
)
```

### 4.2 실시간 카메라 미리보기 (cv4\_3.py)

`while` 루프를 돌며 매 프레임을 `capture_array()`로 가져와 상하 반전 처리 후 OpenCV 창에 표시합니다. ESC 키(ASCII 27)를 누르면 루프가 종료됩니다.

```
# cv4_3.py
from picamera2 import Picamera2
from libcamera import Transform
import cv2

picam2 = Picamera2()
preview_config = picam2.create_preview_configuration(
    main={"format": "XRGB8888", "size": (640, 480)}
)
picam2.configure(preview_config)
picam2.start()                                # 스트리밍 시작

while True:
    image = picam2.capture_array()             # 프레임 → NumPy 배열
    v_image = cv2.flip(image, 0)               # 상하 반전 (0: X축 기준)

    cv2.imshow("CAM Preview", v_image)

    key = cv2.waitKey(30) & 0xFF              # 30ms 대기, 1바이트 마스크
    if key == 27:                             # ESC (ASCII 27) → 종료
        break

cv2.destroyAllWindows()
(venv) $ python3 cv4_3.py
```

#### 4.2.1 코드 상세 해설

**cv2.flip(image, 0):** flipCode 0은 X축(가로선) 기준 상하 반전, 1은 Y축 기준 좌우 반전, -1은 양쪽 동시 반전입니다. 카메라 장착 방향에 따라 값을 조정합니다.

**cv2.waitKey(30) & 0xFF:** 30ms마다 키 입력을 확인합니다. & 0xFF는 일부 환경에서 waitKey 반환값이 음수가 되는 문제를 방지하는 비트 마스크입니다. ESC 코드는 27입니다.

💡 *팁: waitKey(30)은 약 33fps의 루프 속도를 만듭니다. 더 부드러운 미리보기를 원하면 값을 줄이고, CPU 부하를 낮추려면 늘립니다. waitKey(1)은 거의 카메라 최대 프레임레이트로 동작합니다.*

## 4.3 키 입력으로 사진 저장 (cv4\_4.py)

미리보기를 표시하면서 's' 키를 누를 때마다 현재 프레임을 번호를 붙인 파일로 저장하는 예제입니다. 학습 데이터셋 구축, 실험 기록 등에 유용한 패턴입니다.

```
# cv4_4.py
from picamera2 import Picamera2
from libcamera import Transform
import cv2

picam2 = Picamera2()
preview_config = picam2.create_preview_configuration(
    main={"format": "XRGB8888", "size": (640, 480)}
)
picam2.configure(preview_config)
picam2.start()

img_no = 0                                # 저장 파일 번호 카운터

while True:
    image = picam2.capture_array()
    v_image = cv2.flip(image, 0)
    cv2.imshow("CAM Preview", v_image)

    key = cv2.waitKey(30) & 0xFF
    if key == 27:                            # ESC: 종료
        break
    elif key == 115:                         # 's' 키 ASCII 115
        img_no += 1
        filename = "test" + str(img_no) + ".jpg"
        print(filename)
        picam2.capture_file(filename) # Picamera2로 원본 화질 저장

cv2.destroyAllWindows()
(venv) $ python3 cv4_4.py
```

🔗 *picam2.capture\_file()은 Picamera2 내부 버퍼에서 최고 품질로 저장합니다. OpenCV로 처리된 프레임(반전·색상 조정 등 적용 후)을 저장하려면 cv2.imwrite(filename, v\_image)를 사용합니다.*

### 4.3.1 ord() 함수를 이용한 가독성 향상

ASCII 코드 숫자(115, 27) 대신 ord() 함수를 사용하면 코드 의도를 쉽게 파악할 수 있습니다.

```
key = cv2.waitKey(30) & 0xFF
```

```
if key == ord('q'):    # 'q' 키: 종료 (ESC 대신 사용 가능)
    break
elif key == ord('s'):  # 's' 키: 사진 저장
    img_no += 1
    picam2.capture_file(f"test{img_no}.jpg")
elif key == ord('g'):  # 'g' 키: 그레이스케일 전환 (응용 예시)
    gray = cv2.cvtColor(v_image, cv2.COLOR_BGR2GRAY)
    cv2.imshow("CAM Preview", gray)
```

## 제 5장 YOLO v11 설치 및 실시간 객체 탐지

### 5.1 YOLO 개요

YOLO(You Only Look Once)는 이미지 전체를 단 한 번의 신경망 순전파로 처리하여 객체 위치(바운딩 박스)와 클래스를 동시에 예측하는 실시간 객체 탐지 알고리즘입니다. 2024년 Ultralytics가 발표한 YOLO11은 이전 YOLOv8 대비 정확도와 속도를 모두 향상시켰습니다.

본 실습에서는 YOLO11 모델 중 파라미터 수가 가장 적은 YOLO11n(nano) 모델을 사용합니다. YOLO11n은 약 2.6M 파라미터로 매우 가벼워 RPi5 같은 임베디드 환경에서도 동작하며, COCO 데이터셋의 80가지 클래스(사람, 자동차, 의자, 스마트폰 등)를 사전 학습으로 탐지합니다.

| 모델      | 파라미터  | mAP(COCO) | 특징             |
|---------|-------|-----------|----------------|
| YOLO11n | 2.6M  | 39.5      | 최소·최속, RPi5 권장 |
| YOLO11s | 9.4M  | 47.0      | 소형             |
| YOLO11m | 20.1M | 51.5      | 중형             |
| YOLO11l | 25.3M | 53.4      | 대형             |
| YOLO11x | 56.9M | 54.7      | 최고 정확도         |

### 5.2 YOLO v11 설치

#### 5.2.1 의존성 시스템 라이브러리

PyTorch 설치 전에 필요한 시스템 라이브러리를 먼저 설치합니다.

```
(venv) $ sudo apt-get -y install python3-pip libjpeg-dev \
libopenblas-dev libopenmpi-dev libomp-dev
```

| 패키지             | 역할                                    |
|-----------------|---------------------------------------|
| libjpeg-dev     | JPEG 인코딩/디코딩 라이브러리 (OpenCV, PIL 의존)   |
| libopenblas-dev | 수치 선형대수 BLAS 라이브러리 (PyTorch 행렬 연산 가속) |
| libopenmpi-dev  | MPI 병렬 연산 인터페이스 (PyTorch 분산 지원)       |
| libomp-dev      | OpenMP 멀티스레드 병렬 처리 (멀티코어 CPU 활용)      |

#### 5.2.2 PyTorch 및 Ultralytics 설치

PyTorch는 YOLO11의 핵심 딥러닝 프레임워크이고, ultralytics 패키지에 YOLO11 모델 정의·추론 코드가 포함되어 있습니다.

```
(venv) $ pip3 install torch torchvision torchaudio ultralytics
```

⚠ RPi5용 PyTorch는 CPU 전용입니다. GPU(CUDA) 가속은 지원되지 않으므로 추론 속도가 PC 환경보다 느립니다. 이후 NCNN 변환 과정을 통해 RPi5에서의 속도를 크게 개선할 수 있습니다.

## 5.3 YOLO11 모델 정보 확인

설치 완료 후 사전 학습된 yolo11n.pt 모델을 로드하여 탐지 가능한 80개 클래스 목록을 확인합니다. 최초 실행 시 yolo11n.pt 파일이 자동으로 다운로드됩니다(약 5.4MB).

```
# yolov11_model_names.py
from ultralytics import YOLO

model = YOLO("yolo11n.pt")          # 최초 실행 시 자동 다운로드

print(f'탐지 가능 클래스 수: {len(model.names)}')
print('클래스 목록:')
for idx, name in model.names.items():
    print(f' {idx:3d}: {name}')

(venv) $ python3 yolov11_model_names.py
```

출력 결과에서 person(0), bicycle(1), car(2) 등 80개 클래스 목록을 확인할 수 있습니다. 이 모델은 이 클래스에 속하는 객체를 별도 학습 없이 즉시 탐지할 수 있습니다.

## 5.4 NCNN 모델 변환

NCNN(Neural Network Computing Framework)은 Tencent가 개발한 모바일·임베디드 특화 경량 딥러닝 추론 프레임워크입니다. PyTorch .pt 모델을 NCNN 포맷으로 변환하면 ARM CPU에서의 추론이 최적화되어 RPi5에서 속도가 크게 향상됩니다.

### 5.4.1 NCNN 변환 스크립트 (convert2ncnn.py)

```
# convert2ncnn.py
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

# NCNN 포맷으로 변환 (입력 크기 640x640)
model.export(format="ncnn", imgsz=(640, 640))

(venv) $ python3 convert2ncnn.py
```

변환이 완료되면 현재 디렉토리에 yolo11n\_ncnn\_model/ 폴더가 생성됩니다. 이 폴더 안의 .param(네트워크 구조)과 .bin(가중치) 파일이 NCNN 모델입니다.

 한 줄 명령으로도 변환 가능합니다.

```
(venv) $ python -c 'from ultralytics import YOLO;
YOLO("yolo11n.pt").export(format="ncnn")'

# 또는 CLI
(venv) $ yolo export model=yolo11n.pt format=ncnn
```

## 5.5 NCNN 실시간 객체 탐지 프로그램

변환된 NCNN 모델로 Picamera2 라이브 영상에서 실시간으로 객체를 탐지합니다. 탐지된 객체마다 바운딩 박스와 클래스 레이블이 표시되고, 화면 좌상단에 초당 프레임 수(FPS)가 실시간으로 표시됩니다.

### 5.5.1 FPS 측정 버전 (yolov11ncnn-test.py)

```
# yolov11ncnn-test.py
import cv2
import time
from ultralytics import YOLO
from picamera2 import Picamera2

# — 카메라 초기화 —————
picam2 = Picamera2()
picam2.preview_configuration.main.size = (640, 480)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.align()
picam2.configure("preview")
picam2.start()

# — NCNN 모델 로드 —————
model = YOLO("yolo11n_ncnn_model")

prev_time = time.time()

while True:
    # 1. 프레임 캡처
    frame = picam2.capture_array()

    # 2. FPS 계산
    current_time = time.time()
    elapsed_time = current_time - prev_time
    prev_time = current_time
    fps = 1 / elapsed_time if elapsed_time > 0 else 0.0

    # 3. YOLO 객체 탐지 실행
    results = model(frame)

    # 4. 탐지 결과 시각화 (바운딩 박스 + 레이블 자동 그리기)
    annotated_frame = results[0].plot()

    # 5. 탐지 클래스를 터미널에 출력
    for result in results:
        for box in result.boxes:
            print(result.names[int(box.cls[0])])

    # 6. FPS 텍스트 오버레이
    cv2.putText(annotated_frame, f"FPS: {fps:.2f}",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2, cv2.LINE_AA)

    # 7. 화면 표시 및 종료 처리
    cv2.imshow("YOLO Real-time Detection", annotated_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()
(venv) $ python3 yolov11ncnn-test.py
```

## 5.5.2 핵심 코드 해설

**picam2.preview\_configuration.main.format = "RGB888"**: YOLO 모델은 RGB 입력을 기대합니다. 이 포맷으로 설정하면 `capture_array()` 반환값이 RGB 순서 배열이므로 별도 색 공간 변환 없이 모델에 전달할 수 있습니다.

**model(frame)**: NumPy 배열 프레임을 전달하면 **Results** 리스트를 반환합니다. NCNN 모델의 경우 RPi5에서 약 3~7 FPS 수준으로 동작합니다.

**results[0].plot()**: 탐지된 모든 객체에 바운딩 박스, 클래스 레이블, 신뢰도 점수를 그려 넣은 프레임 (NumPy 배열)을 반환합니다. 화면 표시용으로 가장 간편한 방법입니다.

**box.cls[0], box.conf[0], box.xyxy[0]**: 각각 탐지 객체의 클래스 인덱스, 신뢰도(0~1), 바운딩 박스 좌표 (x1,y1,x2,y2)입니다.

## 5.5.3 간소화 버전

FPS 계산·터미널 출력을 제거한 최소 구조의 예제입니다. 빠른 테스트와 코드 구조 이해에 적합합니다.

```
# 간소화 버전
import cv2
from picamera2 import Picamera2
from ultralytics import YOLO

picam2 = Picamera2()
picam2.preview_configuration.main.size = (1280, 720)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.align()
picam2.configure("preview")
picam2.start()

model = YOLO("yolo11n_ncnn_model")

while True:
    frame = picam2.capture_array()
    results = model(frame)
    annotated_frame = results[0].plot()
    cv2.imshow("YOLO Detection", annotated_frame)
    if cv2.waitKey(1) == ord("q"): break

picam2.stop()
cv2.destroyAllWindows()
```

## 5.5.4 탐지 결과 상세 추출

**results** 객체에서 각 탐지 객체의 세부 정보(클래스, 신뢰도, 좌표)를 직접 추출하는 방법입니다.

```
results = model(frame)
result = results[0]

for box in result.boxes:
    cls_id = int(box.cls[0]) # 클래스 인덱스 (0~79)
```



```

cls_name = result.names[cls_id] # 클래스 이름 ('person' 등)
conf      = float(box.conf[0])   # 신뢰도 (0.0~1.0)
x1,y1,x2,y2 = box.xyxy[0].tolist() # 바운딩 박스 (픽셀 좌표)

if conf > 0.5: # 신뢰도 50% 이상만 출력
    print(f"{cls_name}: {conf:.2f} [{x1:.0f},{y1:.0f}~{x2:.0f},{y2:.0f}]")

```

## 5.6 PyTorch 모델 vs NCNN 모델 성능 비교

동일한 YOLO11n 모델을 PyTorch .pt 방식과 NCNN 변환 방식으로 RPi5에서 실행하면 아래와 같은 차이가 있습니다.

| 항목       | PyTorch (.pt) | NCNN 변환 후           | 비고        |
|----------|---------------|---------------------|-----------|
| 추론 FPS   | 0.5 ~ 1.5 FPS | 3 ~ 7 FPS           | 3~5배 향상   |
| 메모리 사용량  | 약 400MB       | 약 120MB             | 대폭 감소     |
| 모델 로드 시간 | 10 ~ 30초      | 2 ~ 5초              | 훨씬 빠름     |
| 파일 크기    | 5.4MB (.pt)   | 2.7MB (.param+.bin) | 절반 수준     |
| 정확도(mAP) | 기준값           | 근소하게 낮음             | 실용상 무시 가능 |

🔗 RPi5 실시간 영상 처리에는 NCNN 변환이 필수입니다. PyTorch .pt 모델은 개발·디버깅용으로만 사용하고, 배포·실습 환경에서는 항상 NCNN 모델을 사용하십시오.

## 5.7 전체 실습 흐름 정리

이 장에서 다룬 YOLO v11 실시간 탐지 실습의 전체 흐름은 다음과 같습니다.

| 단계 | 내용                                                         |
|----|------------------------------------------------------------|
| 1  | 의존 라이브러리 설치 (libjpeg, libopenblas, libopenmpi, libomp)     |
| 2  | pip3 install torch torchvision torchaudio ultralytics      |
| 3  | yolov11_model_names.py 실행 → yolo11n.pt 자동 다운로드 + 클래스 목록 확인 |
| 4  | convert2ncnn.py 실행 → yolo11n_ncnn_model/ 폴더 생성             |
| 5  | yolov11ncnn-test.py 실행 → Picamera2 + NCNN 실시간 탐지 확인        |
| 6  | 'q' 키 입력 → 프로그램 종료                                         |

## 참고 문헌

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤서북, 2024 — Ch09.

이현·김경연·김명생, 라즈베리파이5로 구현하는 사물인터넷(IoT)과 인공지능(AI) 자율주행 자동차, 광문각, 2025 — Ch06, Ch07, Ch13.

🔗 참고: <https://www.perplexity.ai/search/rpi5-bookwormeseo-libcamera-my-bt0L39skQheVvsY0rSojKQ#0>

 참고: <https://www.perplexity.ai/search/rajeuberipai5-bookworm-eseo-yo-5liZyh8oTqCFSu7pyYWyKg>

중앙대학교 전자전기공학부